

From Ambiguous Prose to a Verified Invariant: A Leader-Completeness Bug in the CCF TLA⁺ Specification and Its Commit-Certificate Fix

Ovidiu Marcu^{*}

Claude[†]

Abstract

Natural-language descriptions of distributed consensus protocols are inherently ambiguous: the same English sentence admits several non-equivalent formalizations, and the differences surface only in rare executions. A formal specification removes that ambiguity by forcing a single precise reading—but the *auxiliary invariants* a specification carries to express its safety properties are themselves conjectures, and they can be subtly wrong even when the modeled protocol is correct. We report such a case in the publicly available TLA⁺ specification of Microsoft’s Confidential Consortium Framework (CCF). Its `LeaderCompletenessInv` invariant—a faithful-looking rendering of Raft’s Leader Completeness property—is *not* an invariant of the specification: we exhibit a concrete, reachable 37-step execution that violates it while every other safety invariant, including the core log-agreement property, holds. The defect is a formalization ambiguity, not a protocol bug. We trace it to a single modeling decision—keying the property on each entry’s *authoring* term, read from current state, instead of the term in which the entry was *committed*—and fix it by adding a `committed` commit-certificate history variable that records the commit term, then re-keying Leader Completeness on it. We validate the repair by model checking and publish the original specification (`ccf/`) and the fixed layer (`ccf_fix/`) for independent reproduction.

1 Introduction

Raft’s Leader Completeness property is stated in the Raft work (Ongaro & Ousterhout) as Figure 3:

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms.

The sentence is clear to a human, yet it hides a choice that only a formal reading must resolve: *which* term does “a given term” name—the term in which the entry was **authored**, or the term in which it became **committed**? For *directly* committed entries the two coincide. For *indirectly* committed entries they do not, and they disagree precisely on the executions that make consensus hard.

This is the kind of ambiguity that formal specification exists to eliminate. But specification alone is not enough: a spec can model the protocol’s *transitions* faithfully while still asserting an *invariant* that is a slightly-too-strong reading of the prose. Only verification—here, directed model checking—exposes the gap. We argue, through a concrete case study in a production-grade specification, that (i) prose ambiguity is resolved only by formalization, and (ii) the correctness of a specification’s stated invariants must itself be established by mechanical checking; a plausible, well-reviewed invariant can be false.

2 Background and Related Work

The Confidential Consortium Framework (CCF). CCF is Microsoft’s open-source framework for confidential, highly-available services on a permissioned ledger replicated by a Raft-derived consensus protocol (<https://github.com/microsoft/ccf>). CCF’s consensus is specified in TLA⁺ and maintained alongside the implementation (`tlc/consensus/ccraft.tla`). The specification is an exemplary engineering artifact: it models reconfiguration, signature transactions, pre-vote, term propagation, and a committable-log notion, and ships a substantial suite of invariants and model-checking harnesses. Our work builds directly on that specification and on the broader CCF effort (Howard et al., *Confidential Consortium Framework: Secure Multiparty Applications with Confidentiality, Integrity, and High Availability*). Our contribution is narrow and complementary: a correction to one auxiliary invariant, found and validated with the project’s own tooling (TLA⁺/TLC).

Raft and TLA⁺. The protocol is Raft (Ongaro & Ousterhout, *In Search of an Understandable Consensus Algorithm*). The modeling language is TLA⁺ with the TLC model checker (Lamport). Our analysis is entirely at the specification level.

3 Raft Leader Completeness, the Prose, and Its Keyed Formalization

Leader Completeness is the linchpin of Raft safety: it is what lets a freshly elected leader serve reads and overwrite followers without losing committed data. Raft secures it with two rules. (§5.4.1, *election restriction*) a voter grants

^{*}Formal Systems, Luxembourg.

[†]Anthropic; coauthor.

its vote only to a candidate whose log is at least as up-to-date as its own; (§5.4.2, *commit restriction*) a leader marks an entry committed only once a quorum stores it **and** the entry is from the leader’s **current** term. The second rule is the source of the ambiguity. A leader never commits an old-term entry by counting replicas directly; it commits old entries **indirectly**, by committing a current-term entry on top of them. This is the celebrated Figure-8 scenario: an entry authored in term T_e can remain uncommitted across intervening terms and become committed only later, in term $T_c > T_e$, when a current-term entry above it is committed.

Now reread Figure 3. “Committed in a given term”—is “a given term” T_e (the entry’s own term) or T_c (the term in which it was committed)? For a direct commit $T_e = T_c$ and the question is moot. For an indirect commit $T_e < T_c$, and the two readings make genuinely different demands on the *intermediate* leaders whose term lies strictly between T_e and T_c . The correct reading is the commit term T_c : an entry committed in T_c need only appear in leaders of terms $> T_c$; an intermediate leader at term T with $T_e \leq T < T_c$ is, by the prose, *exempt*. Choosing the entry term T_e instead silently strengthens the property—and, as we show, breaks it.

The natural way to formalize Figure 3 reads the deciding term *directly out of the current log*, with no auxiliary state. This is how a widely-used Raft model-checking specification, `raft.tla`, states Leader Completeness—and, with only cosmetic renaming, how CCF’s `ccraft.tla` states it too:

```

/* raft.tla -- keyed on the ENTRY term, with
/* NO committed history variable.
CommittedTermPrefix(j, x) ==
... SubSeq(log[j], 1,
           min(maxTermIndex, commitIndex[j]))
where maxTermIndex = largest index y with
           log[j][y].term <= x  /* the entry's term
LeaderCompletenessInv ==
  \A i \in Server : state[i] = Leader =>
    \A j \in Server : i /= j =>
      CheckIsPrefix(CommittedTermPrefix(j,
                                         currentTerm[i]), log[j])

```

We call this the **keyed strategy**: the antecedent is keyed on a term value read *directly from current state*—`log[j][y].term`, compared against `currentTerm[i]`—so it needs no history variable, which is its appeal. The specification’s comment is careful and correct about the *universe*: it deliberately includes a stale leader “already [superseded] without the old leader stepping down yet.” The flaw is only the *term*. With $x = \text{currentTerm}[i]$, the invariant demands that leader i at term T contain every committed entry of j whose **entry-term** is $\leq T$ —the entry-term reading T_e , used as a proxy for the commit term T_c . The proxy is exact for direct commits and wrong for indirect ones ($T_e < T_c$): it obligates an intermediate leader, at a term $T_e \leq T < T_c$, to hold an entry that the prose exempts. CCF inherits this keyed strategy verbatim.

4 The Commit-Certificate Strategy

The faithful alternative records the commit term *when a commit happens*, rather than re-deriving it from the log afterward. Ongaro’s reference Raft TLA⁺ specification, extended with a **committed** history variable, does exactly this: at the point a leader advances its commit index it appends one **commit certificate** per newly-committed index, freezing the commit term:

```

/* AdvanceCommitIndex(i): one commit cert per
/* newly-committed index. cterm is the committing
/* leader's current term (= the entry's term, by the
/* commit guard); cquorum is the agreement quorum.
committed' = committed \cup
{ [ index    |-> idx,
  entry     |-> log[i][idx],
  cterm     |-> currentTerm[i], /* COMMIT term
  cleader   |-> i,
  cquorum   |-> Agree(idx) ]
  : idx \in (commitIndex[i]+1)..newCommitIndex }

```

Leader Completeness is then keyed on the recorded commit term `c.term`, over the history of elections:

```

/* Any entry committed in term t appears in the log
/* of every leader of any term > t (Figure 3),
/* COMMIT-term-keyed.
LeaderCompleteness ==
  \A c \in committed :
    \A e \in elections :
      c.term < e.eterm =>
        /\ c.index \in 1..Len(e.elog)
        /\ e.elog[c.index] = c.entry

```

Because `cterm` is captured at commit time as the committing leader’s term, it is the true T_c even for an indirect commit (where the entry’s own term T_e is smaller). The keyed proxy disappears: the invariant now says exactly what Figure 3 says—“committed in term `c.term`, present in every leader of a higher term.” The price is one ghost variable, present only in the specification, that no implementation would carry. §6 applies this strategy to CCF.

5 The Bug: A Reachable Violating Execution

We constructed a directed witness against `ccraft.tla` (3 servers, single static configuration, pre-vote disabled—an initial state the specification’s own harness admits). Every step is a genuine `ccraft` action driven in a fixed order from the real initial state; messages are produced by the spec, not fabricated. The execution (`StartTerm = 2`):

1. **n1** (leader, term 2) appends a client entry **e2** at index 5 and signs it (**s2**, index 6)—but does **not** replicate it.
2. **n2** times out and wins term 3 (because **n3** still holds only the start log, the committable up-to-date check permits it), then appends and signs its own entry **e3** at index 5—**n2**’s log now **diverges** at index 5.

3. **n1** (stepped down to follower by **n2**'s vote request) times out and wins term 4 with **n3**; on becoming leader it truncates to its committable prefix, which **keeps e2/s2**. It appends **e4/s4** and replicates indices 5–8 to **n3**.

4. **n1** advances its commit index to the term-4 signature at index 8, which **indirectly commits** the term-2 entry **e2** at index 5.

In the final state, `commitIndex[n1] = 8`, while **n2** is still a leader at term 3 with the term-3 entry at index 5. Evaluating the keyed `LeaderCompletenessInv` at the stale leader **n2**: `CommittedTermPrefix(n1, currentTerm[n2]=3)` includes index 5, because the entry there has term $2 \leq 3$ —so the invariant demands **n2** contain **e2**. It does not. **The invariant is violated.** This is the §3 ambiguity made concrete: **e2**'s authoring term is 2, its commit term is 4, and $2 \leq 3 < 4$.

Crucially, this is *not* a safety violation of the protocol. CCF's committed-log agreement invariant (`LogInv`) and the quorum and log-matching invariants all hold throughout—**n2** never *committed* its divergent entry. The bug lives entirely in the auxiliary invariant's term keying.

Two observations explain why the defect persisted in a heavily-checked specification. First, the violation requires at least three terms; the project's harness defaults to a single term, where the entry-term and commit-term readings coincide. Second, the witness is roughly 37 steps deep—beyond breadth-first or random search (in our experiments, exhaustive 3-node breadth-first search reached only depth $\sim 19 / 45\text{M}$ states, and random simulation explored over seven million states, neither reaching the configuration). The bug is real but lies in a region only a directed search visits. The absence of a counterexample under default bounds is not evidence of invariance.

6 The Fix for CCF

We apply the commit-certificate strategy of §4 to CCF, layered over the untouched upstream specification (`ccffix.tla`). CCF commits originate only at a leader's commit advance, so we record certificates exactly there, detecting the step from the observed state change (so `committed` is a pure ghost and no upstream action is edited):

```

LeaderCommitters ==
  { i \in Servers : /\ leadershipState[i]=Leader
                    /\ commitIndex'[i]>commitIndex[i]
                    /\ log'[i] = log[i] }
LeaderCommitCerts ==
  UNION { { [ index|->idx, entry|->log[i][idx],
            cterm|->currentTerm[i], \* COMMIT term
            cleader|->i ]
          : idx \in (commitIndex[i]+1)
            ..commitIndex'[i] }
        : i \in LeaderCommitters }
FixNext == Next /\ committed' =
  committed \cup LeaderCommitCerts

```

CCF records no election history, so the “every higher-term leader” universe is the set of current leaders—the same universe the original invariant uses—and the property becomes:

```

LeaderCompleteness ==
  \A s \in Servers : leadershipState[s]=Leader =>
  \A c \in committed :
    c.cterm < currentTerm[s] => \* COMMIT term
    /\ c.index \in DOMAIN log[s]
    /\ log[s][c.index] = c.entry

```

On the witness of §5, the certificate for index 5 carries `cterm = 4`, so for the stale leader **n2** at term 3 the antecedent $4 < 3$ is false and **n2** is correctly exempt. The execution that broke the keyed invariant satisfies the commit-certificate invariant.

7 Validation by Model Checking

We validated the repair with the project's tooling (TLA^+/TLC). (i) **Before/after on the exact witness.** The scripted 37-step execution violates the original `LeaderCompletenessInv` and satisfies the fixed `LeaderCompleteness`—identical state sequence, opposite verdicts. (ii) **Bounded breadth-first search** from the real initial state (3 servers, terms up to 4) explores millions of states with no violation of the fixed invariant, `LogInv`, or the certificate type invariant. (iii) **Simulation** of deep random executions (depth 120, well past the witness depth) checked millions of states with no violation. The history variable grows monotonically, so exhaustive search does not terminate; the evidence is therefore strong but bounded.

8 Discussion

Two lessons generalize beyond CCF. *First*, prose ambiguity in protocol descriptions is resolved only by formalization: “committed in a given term” has two formal readings, and no amount of careful English distinguishes them—the model checker does. *Second*, a specification's invariants are claims that must themselves be verified; the entry-term keying is a reasonable, widely-used, well-reviewed formalization that is nonetheless false, and only mechanical checking surfaced the gap. The remedy generalizes too: when a property refers to *when* something happened (here, the commit term), record it with a history variable rather than reconstruct it from current state—the commit-certificate strategy of §4. The change is local, leaves the modeled protocol and its core safety properties untouched, and brings the stated invariant into agreement with the prose it is meant to capture.

We stress that this is a correction to an auxiliary invariant in a specification, not a vulnerability in CCF or in the Raft protocol: the protocol's committed-log agreement holds throughout the witnessing execution. The CCF

specification’s breadth and its open development made this analysis possible.

9 Availability

The original specification and harness (`ccf/`) and the fixed layer with all model-checking drivers (`ccf_fix/`) are published alongside this paper. CCF itself is open source at <https://github.com/microsoft/ccf>. All results are reproducible with TLA⁺/TLC; commands are in `ccf_fix/fix.readme`.

References

- [1] H. Howard et al. *Confidential Consortium Framework: Secure Multiparty Applications with Confidentiality, Integrity, and High Availability*. Microsoft Research. Microsoft CCF: <https://github.com/microsoft/ccf>.
- [2] D. Ongaro and J. Ousterhout. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC 2014; and D. Ongaro, Ph.D. dissertation (Leader Completeness, Figure 3; the Figure-8 indirect-commit scenario, §5.4).
- [3] L. Lamport. *Specifying Systems: The TLA⁺ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.